ShareBox: Code Documentation

Name: Harshiv Bharat Patel

Student Number: 3120107

GitHub: https://github.com/harshivpatel/ShareBox

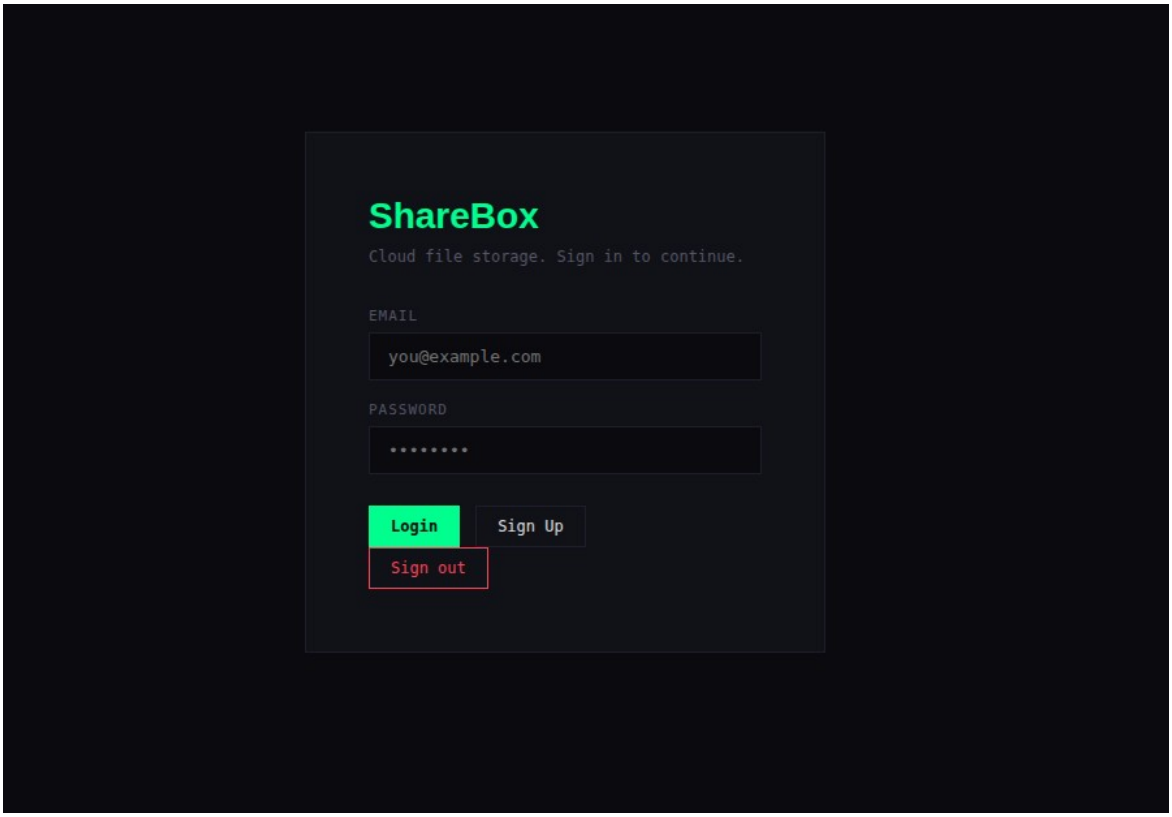
Note on commits: The application was developed locally first and Git was introduced toward the end of development. All commits were made within a short window as a result, rather than incrementally during development.

1. General Approach

ShareBox is a simplified cloud storage application built using FastAPI as the web framework, Google Cloud Firestore as the database, Google Cloud Storage for file storage, and Firebase Authentication for user login. The application follows a server-rendered architecture using Jinja2 templates, where all business logic resides in main.py and the frontend is a set of HTML templates styled with CSS.

The application is structured around three core Firestore collections: users, directories, and files. When a user logs in for the first time, a user document and a root directory are created automatically. From there, users can navigate directories, upload and download files, share files with other registered users, and detect duplicate files both locally and globally across their entire storage.

Navigation state is maintained using browser cookies, specifically the current_path cookie which tracks which directory the user is currently viewing. All destructive operations such as deletions use Firestore document IDs rather than names to prevent accidental operations on the wrong document.



2. Helper Functions

get_user(user_token)

Queries the Firestore users collection using the user ID from the verified Firebase token. Returns the document snapshot, which is checked for existence to determine if this is a first-time login.

create_user(user_token)

Called when a user logs in for the first time. Creates a document in the users collection storing the user's email, and creates a corresponding root directory document in the directories collection with path set to / and parent set to None.

get_directories(user_id, current_path)

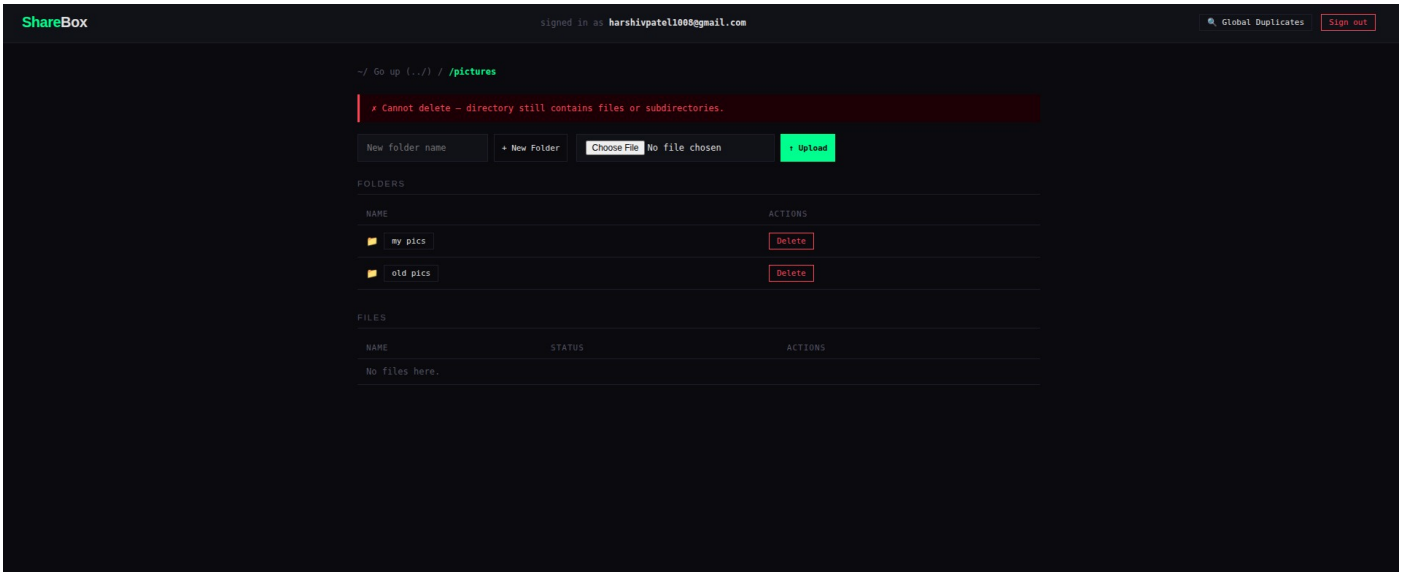
Fetches all directory documents from Firestore where the owner matches the current user and the path matches the current directory. Returns a list of dictionaries containing the document ID and directory name, which are used to render the folder list in the template.

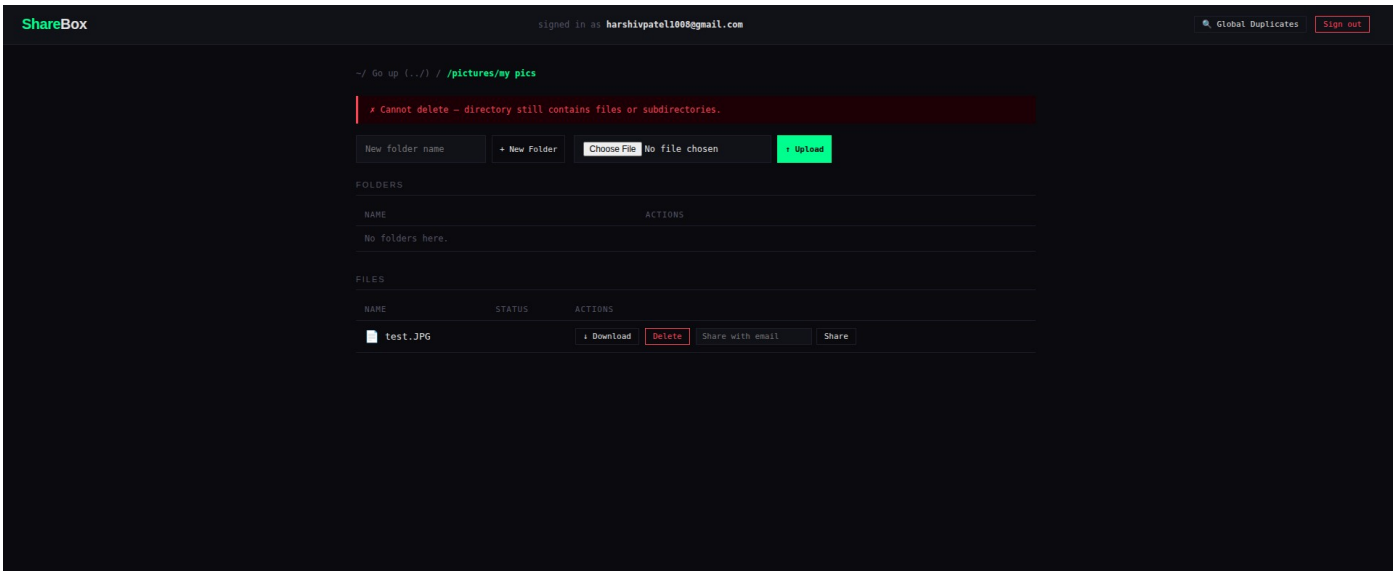
get_files(user_id, current_path)

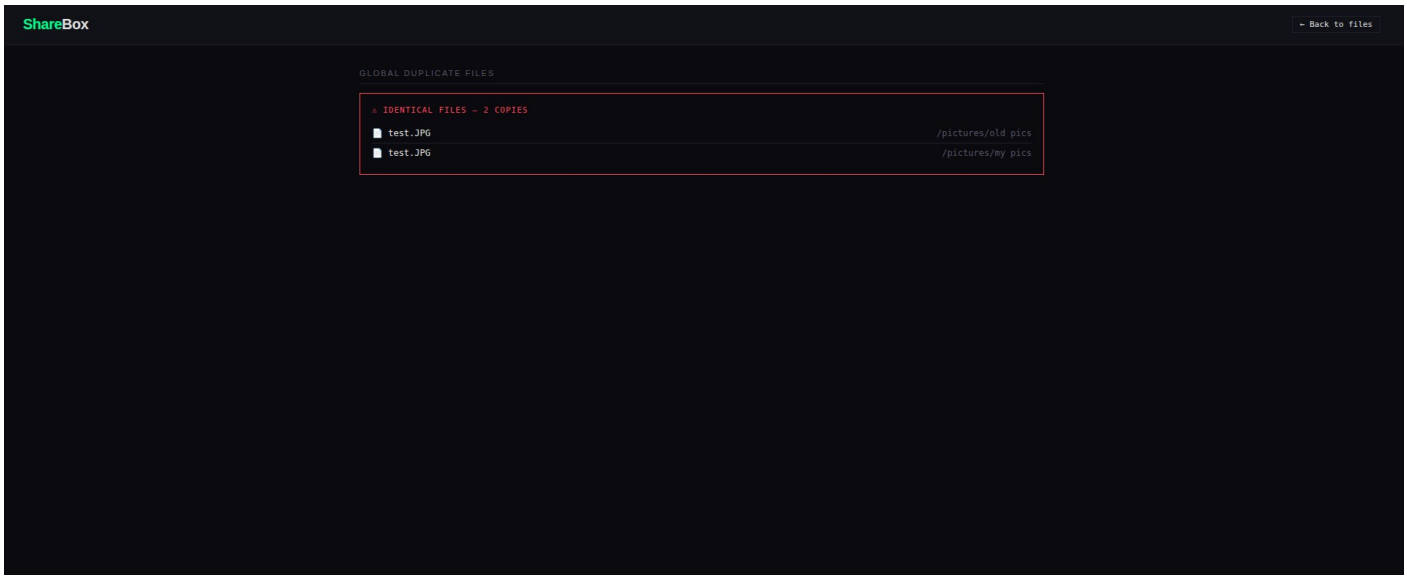
Fetches all files owned by the user in the current directory, as well as any files shared with the user via the shared_with array field. Each file entry includes its ID, name, hash, and whether the current user is the owner. After collecting all files, the function groups them by MD5 hash to identify duplicates and marks each file with an is_duplicate flag. This function powers both the file listing and the local duplicate detection feature.

get_all_duplicates(user_id)

Fetches all files owned by the user across all directories and groups them by MD5 hash. Any group containing more than one file is considered a duplicate set. Returns a list of groups, where each group is a list of file objects containing the name and path, allowing the user to see where duplicate content exists across their entire storage.


ShareBox
signed in as harshivpatel1008@gmail.com
Global Duplicates
Sign out
~/ Go up (../) / /pictures
✗ Cannot delete — directory still contains files or subdirectories.
New folder name
+ New Folder
Choose File No file chosen
↑ Upload
FOLDERS
NAME
ACTIONS
my pics
Delete
old pics
Delete
FILES
NAME
STATUS
ACTIONS
No files here.


ShareBox
signed in as harshivpatel1008@gmail.com
Global Duplicates
Sign out
~/ Go up (../) / /pictures/my pics
✗ Cannot delete — directory still contains files or subdirectories.
New folder name
+ New Folder
Choose File No file chosen
↑ Upload
FOLDERS
NAME
ACTIONS
No folders here.
FILES
NAME
STATUS
ACTIONS
test.JPG
↓ Download
Delete
Share with email
Share


ShareBox
← Back to files
GLOBAL DUPLICATE FILES
⚠ IDENTICAL FILES — 2 COPIES
test.JPG
/pictures/old pics
test.JPG
/pictures/my pics

3. Route Handlers

GET / — root

The main route that renders the dashboard. Verifies the Firebase token from the browser cookie, checks if the user exists in Firestore and creates them if not, then fetches the directory and file listings for the current path. Passes all data to the main.html template for rendering.

POST /navigate — navigate

Handles clicking into a subdirectory. Reads the current path from the cookie and appends the selected directory name to build the new path. Sets the updated path as a cookie and redirects back to the root route.

GET /navigate-up — navigate_up

Handles the go up action. Trims the last segment from the current path using rsplit to move up one level. If the result is an empty string, it defaults back to the root path. Updates the cookie and redirects.

POST /create-directory — create_directory

Creates a new directory in the current path after verifying the user token. Before creating, it queries Firestore to check if a directory with the same name already exists in the same location. If a duplicate is found, the request is silently ignored. This prevents the major bug of having two directories with the same name in the same location.

POST /delete-directory — delete_directory

Deletes a directory by its Firestore document ID. Before deleting, it checks whether the directory is the root directory and blocks that operation. It also queries for any files or subdirectories whose path matches the full path of the directory being deleted. If any exist, the deletion is blocked and an error cookie is set to inform the user. This satisfies the requirement to prevent deletion of non-empty directories.

POST /upload-file — upload_file

Handles file uploads. Reads the file contents into memory and computes an MD5 hash for duplicate detection. Checks if a file with the same name exists in the current directory. If it does and the user has not confirmed overwrite, a cookie is set to trigger the overwrite confirmation banner in the template. If overwrite is confirmed, the old file is deleted from both GCS and Firestore before the new one is uploaded. The file is stored in GCS using a blob path constructed from the user ID and current path, and a corresponding document is saved to the files Firestore collection including the hash.

POST /clear-overwrite — clear_overwrite

Clears the overwrite confirmation cookie when the user clicks Cancel on the overwrite banner. Redirects back to the root route.

POST /delete-file — delete_file

Deletes a file by its Firestore document ID. Retrieves the document to get the GCS blob path, deletes the blob from Cloud Storage, then deletes the Firestore document. Using the document ID prevents accidental deletion of the wrong file.

GET /download-file/{file_id} — download_file

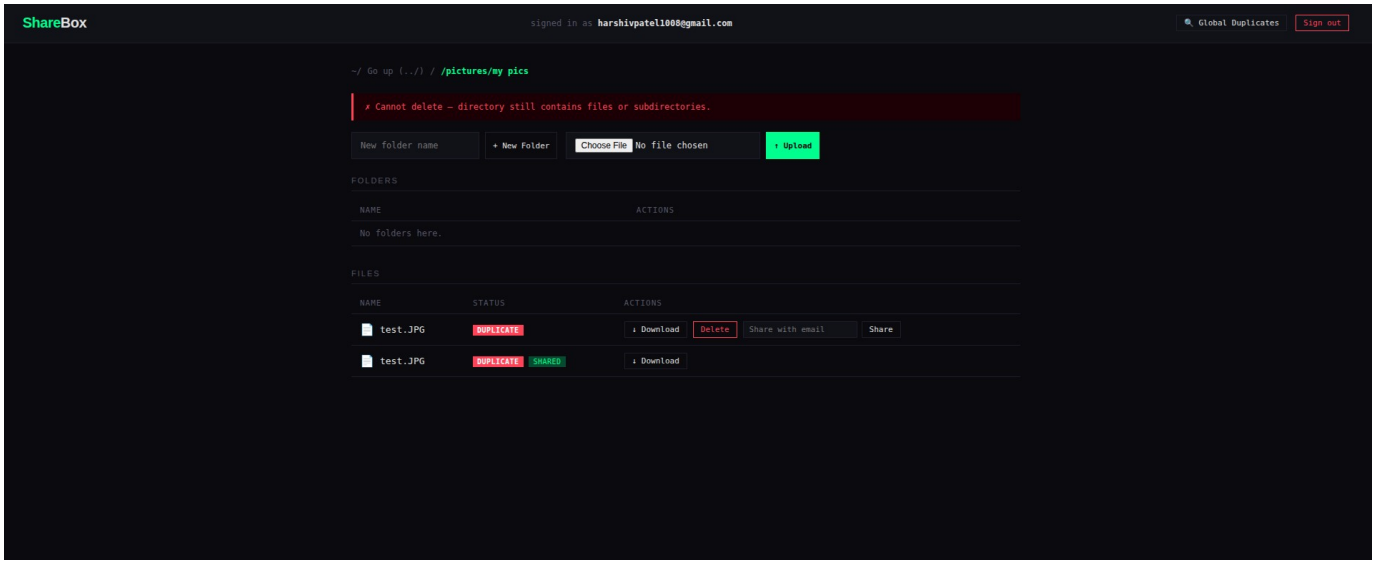
Downloads a file by streaming it directly from GCS to the browser. Retrieves the blob path from Firestore, downloads the blob into a BytesIO buffer, and returns a StreamingResponse with a Content-Disposition header so the browser treats it as a file download. This approach avoids the need for signed URLs which require a service account key.

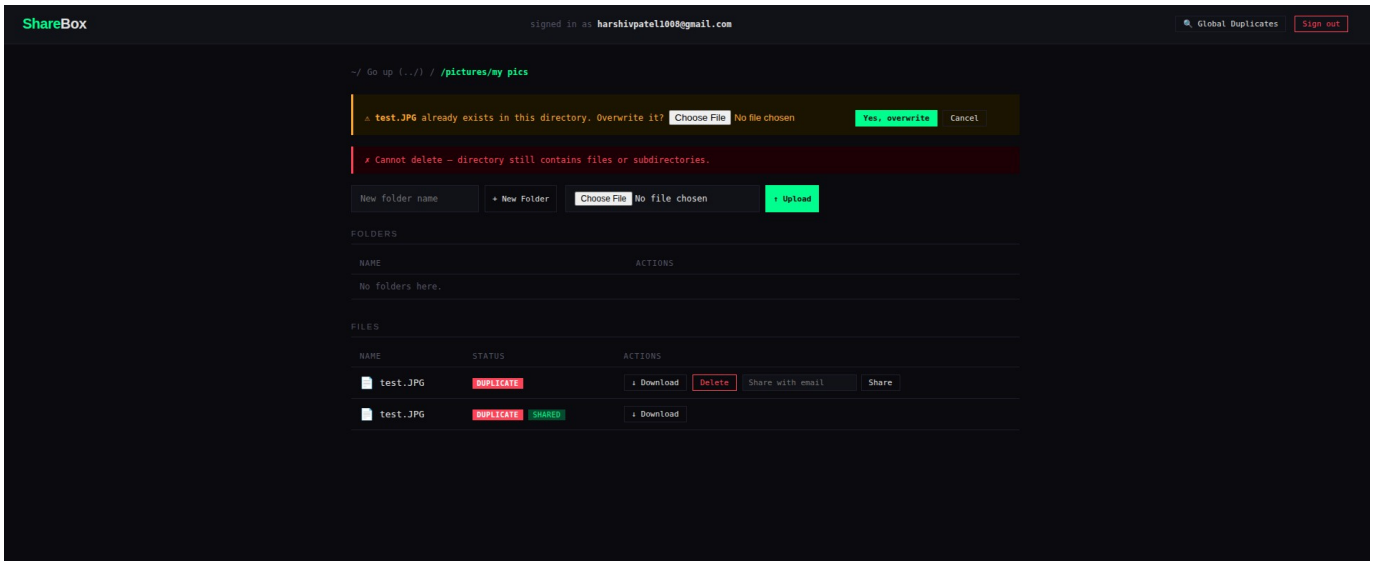
GET /duplicates — duplicates

Renders the global duplicates page. Calls get_all_duplicates to find all files across the user's entire storage that share an MD5 hash. Passes the grouped results to duplicates.html for display.

POST /share-file — share_file

Shares a file with another registered user. Looks up the target user by email in the users Firestore collection. If found, uses Firestore ArrayUnion to add the target user's ID to the shared_with array on the file document. The shared user can then download the file but cannot delete or modify it, as the template checks the is_owner flag before showing those controls.


ShareBox
signed in as harshivpatel1008@gmail.com
Global Duplicates
Sign out
~/ Go up (../) / /pictures/my pics
✗ Cannot delete — directory still contains files or subdirectories.
New folder name
+ New Folder
Choose File No file chosen
↑ Upload
FOLDERS
NAME
ACTIONS
No folders here.
FILES
NAME
STATUS
ACTIONS
test.JPG
DUPLICATE
↓ Download
Delete
Share with email
Share
test.JPG
DUPLICATE
SHARED
↓ Download


ShareBox
signed in as harshivpatel1008@gmail.com
Global Duplicates
Sign out
~/ Go up (../) / /pictures/my pics
⚠ test.JPG already exists in this directory. Overwrite it?
Choose File No file chosen
Yes, overwrite
Cancel
✗ Cannot delete — directory still contains files or subdirectories.
New folder name
+ New Folder
Choose File No file chosen
↑ Upload
FOLDERS
NAME
ACTIONS
No folders here.
FILES
NAME
STATUS
ACTIONS
test.JPG
DUPLICATE
↓ Download
Delete
Share with email
Share
test.JPG
DUPLICATE
SHARED
↓ Download